 How to use Learning Curves to Diagnose Machine Learning Model Performance

[Click to Take the FREE Deep Learning Performance Crash-Course](#)

Search...

Stacking Ensemble for Deep Learning

Neural Networks in Python

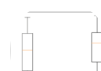
Q

A Gentle Introduction to the Rectified Linear Unit (ReLU)

by **Jason Brownlee** on January 9, 2019 in **Deep Learning Performance**



Last Updated on August 20, 2020

 How to use Data Scaling Improve Deep Learning Model Stability and Performance

neural network, the activation function is responsible for transforming the summed weighted input from the node into the activation of the node or output for that input.

The **rectified linear activation function** or **ReLU** for short is a piecewise linear function that will output the input if it is positive, otherwise, it will output zero. It has become the default activation function for many types of neural networks because a model that uses it is easier to train and often achieves better performance.

In this tutorial, you will discover the rectified linear activation function for deep learning neural networks.

Loving the Tutorials?

After completing this tutorial, you will know:

- The sigmoid and hyperbolic tangent activation functions cannot be used in networks with many layers due to the vanishing gradient problem. >> SEE WHAT'S INSIDE
- The rectified linear activation function overcomes the vanishing gradient problem, allowing models to learn faster and perform better.
- The rectified linear activation is the default activation when developing multilayer Perceptron and convolutional neural networks.

Kick-start your project with my new book [Better Deep Learning](#), including *step-by-step tutorials* and the *Python source code* files for all examples.

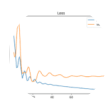
Let's get started.

- **Jun/2019:** Fixed error in the equation for He weight initialization (thanks Maltev).

Never miss a tutorial:



Picked for you:



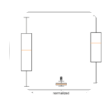
How to use Learning Curves to Diagnose Machine Learning Model Performance



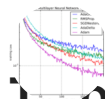
Stacking Ensemble for Deep Learning Neural Networks in Python



How To Improve Deep Learning Performance



How to use Data Scaling Improve Deep Learning Model Stability and Performance



A Gentle Introduction to the Rectified Linear Activation
Photo by Bureau of Land Management
Gentle Introduction to the Adam
Optimization Algorithm for Deep Learning

Tutorial Overview

This tutorial is divided into six parts; they are:

Loving the Tutorials?

1. Limitations of Sigmoid and Tanh Activation Functions
2. Rectified Linear Activation Function
The Better Deep Learning Ebook is where you'll find the Really Good stuff.
3. How to Implement the Rectified Linear Activation Function
4. Advantages of the Rectified Linear Activation Function >> SEE WHAT'S INSIDE
5. Tips for Using the Rectified Linear Activation Function
6. Extensions and Alternatives to ReLU

Limitations of Sigmoid and Tanh Activation Functions

A neural network is comprised of layers of nodes and learns to map examples of inputs to outputs.

For a given node, the inputs are multiplied by the weights in a node and summed together. This value is referred to as the summed activation of the node. The summed activation is then transformed via an [activation function](#) and defines the specific output or “activation” of the node.

The simplest activation function is referred to as the linear activation, where no transform is applied at all. A network comprised of only linear activation functions is very easy to train, but cannot learn complex mapping functions. Linear activation functions are still used in the output layer for networks that predict a quantity (e.g. regression problems).

Nonlinear activation functions are preferred as they allow the nodes to learn more complex structures in the data. Traditionally, two widely used nonlinear activation functions are the **sigmoid** and **hyperbolic tangent** activation functions.

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

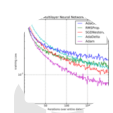
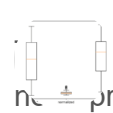
START MY EMAIL COURSE

Start Machine Learning

The sigmoid activation function, also called the logistic function, is traditionally a very popular activation function for neural networks. The input to the function is transformed into a value between 0.0 and 1.0. Inputs that are much larger than 1.0 are transformed to the value 1.0, similarly, values much smaller than 0.0 are snapped to 0.0. The shape of the function for all possible inputs is an S-shape from zero up through 0.5 to 1.0. For a long time, through the early 1990s, it was the default activation used on neural networks.

Picked for you:

The hyperbolic tangent function, or tanh for short, is a similar shaped nonlinear activation function that outputs between -1.0 and 1.0. In the later 1990s and through the 2000s, the tanh function was preferred over the sigmoid activation function as models that used it were easier to train and often had better predictive performance.



... [Stacking Ensemble Models for Deep Learning](#)

... [Neural Networks in Python](#)

— Page 195, [Deep Learning](#), 2016.

... [How To Improve Deep Learning Performance](#)

... [How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)

... [Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

... *Sigmoidal units saturate across most of their domain—they saturate to a high value when z is very positive, saturate to a low value when z is very negative, and are only strongly sensitive to their input when z is near 0.*

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Loving the Tutorials?

— Page 195, [Deep Learning](#), 2016.

The [Better Deep Learning](#) eBook is where you'll find the **Really Good** stuff.

Finally, as the capability of hardware increased through GPUs' very deep neural networks using sigmoid and tanh activation >> SEE WHAT'S INSIDE be trained.

Layers deep in large networks using these nonlinear activation functions fail to receive useful gradient information. Error is back propagated through the network and used to update the weights. The amount of error decreases dramatically with each additional layer through which it is propagated, given the derivative of the chosen activation function. This is called the **vanishing gradient problem** and prevents deep (multi-layered) networks from learning effectively.

“Vanishing gradients make it difficult to know which direction the parameters should move to improve the cost function”

— Page 290, [Deep Learning](#), 2016.

For an example of how ReLU can fix the vanishing gradients problem, see the tutorial:

- [How to Fix Vanishing Gradients Using the Rectified Linear Activation Function](#)

Although the use of nonlinear activation functions allows neural networks to learn complex mapping functions, they effectively prevent the learning algorithm from working with deep networks.

Workarounds were found in the late 2000s and early 2010s using [mini-batch training](#) on modern machines and [layer-wise training](#) or unsupervised pre-training

Start Machine Learning



Want Better Results with Deep Learning?

Take my free 7-day email crash course now (with sample code).

Picked for you:

Click to sign-up and also get a free PDF Ebook version of the course.



How to use Learning Curves to Diagnose Machine Learning Model Performance

Download Your FREE Mini-Course



Stacking Ensemble for Deep Learning Neural Networks in Python

Rectified Linear Activation Function



How To Improve Deep Learning Performance
to use stochastic gradient descent with backpropagation

an activation function is needed that looks and acts like a linear function to capture complex relationships in the data to be learned.



How to use Data Scaling Improve Deep Learning Model Stability and Performance
activation must also provide more sensitivity to the activation

The solution had been bouncing around in the field for some time until 2009 and 2011 shone a light on it.



Gentle Introduction to the Adam Optimization Algorithm for Deep Learning

The solution is to use the rectified linear activation function, or ReL for short.

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

A node or unit that implements this activation function is referred to as a **rectified linear activation unit**, or ReLU for short. Often, networks that use the rectifier function for the hidden layers are referred to as rectified networks.

Loving the Tutorials?

Adoption of the ReLU Deep Learning Book is considered one of the few milestones in the deep learning revolution, e.g. the techniques that now permit the routine development of very deep neural networks.

>> SEE WHAT'S INSIDE



[a] major algorithmic change that has greatly improved the performance of feedforward networks was the replacement of sigmoid hidden units with piecewise linear hidden units, such as rectified linear units.

— Page 226, *Deep Learning*, 2016.

The rectified linear activation function is a simple calculation that returns the value provided as input directly, or the value 0.0 if the input is 0.0 or less.

We can describe this using a simple if-statement:

```
1 if input > 0:
2     return input
3 else:
4     return 0
```

We can describe this function $g()$ mathematically using the $\max()$ function over the set of 0.0 and the input z ; for example:

```
1  $g(z) = \max\{0, z\}$ 
```

The function is linear for values greater than zero, meaning it has a lot of the desirable properties of a linear activation function when training a neural network using backpropagation.

Start Machine Learning

ive

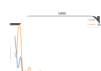
values are always output as zero.

Never miss a tutorial:

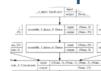


units are nearly linear, they preserve many of the properties that make linear models easy to optimize with gradient-based methods. They also preserve many of the properties that make linear models generalize well.

Picked for you:

 [How to Diagnose Machine Learning Model Performance](#)

Because the rectified function is linear for half of the input domain and nonlinear for the other half, it is referred to as a [piecewise linear function](#) or a hinge function.

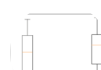
 [Stacking Ensemble for Deep Learning](#)

 [Neural Networks in Python](#)

However, the function remains very close to linear, in that it is composed of two linear pieces.

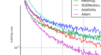
 [How To Improve Deep Learning Performance](#)

Now that we are familiar with the rectified linear activation function, let's see how to use it to improve our models.

 [How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)

How to Code the Rectified Linear Activation Function

We can implement the rectified linear activation function easily using the `max()` function.

 [Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

Here's the simplest implementation is using the `max()` function; for example:

```
1 # rectified linear function
2 def rectified(x):
3     return max(0.0, x)
```

We expect that any positive value will be returned unchanged whereas an input value of 0.0 or a negative value will be returned as the value 0.0.

Below are the inputs and outputs of the rectified linear activation function.

```
1 # demonstrate the rectified linear function
2
3 # rectified linear function
4 def rectified(x):
5     return max(0.0, x)
6
7 # demonstrate with a positive input
8 x = 1.0
9 print('rectified(%.1f) is %.1f' % (x, rectified(x)))
10 x = 1000.0
11 print('rectified(%.1f) is %.1f' % (x, rectified(x)))
12 # demonstrate with a zero input
13 x = 0.0
14 print('rectified(%.1f) is %.1f' % (x, rectified(x)))
15 # demonstrate with a negative input
16 x = -1.0
17 print('rectified(%.1f) is %.1f' % (x, rectified(x)))
18 x = -1000.0
19 print('rectified(%.1f) is %.1f' % (x, rectified(x)))
```

Running the example, we can see that positive values are returned regardless of their size, whereas negative values are snapped to the value 0.0.

```
1 rectified(1.0) is 1.0
2 rectified(1000.0) is 1000.0
3 rectified(0.0) is 0.0
4 rectified(-1.0) is 0.0
```

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Start Machine Learning

5 rectified(-1000.0) is 0.0

Never miss a tutorial:

We can get an idea of the relationship between inputs and outputs of the function by plotting a series of inputs and



The example below generates a series of integers from -10 to 10 and calculates the rectified linear activation for

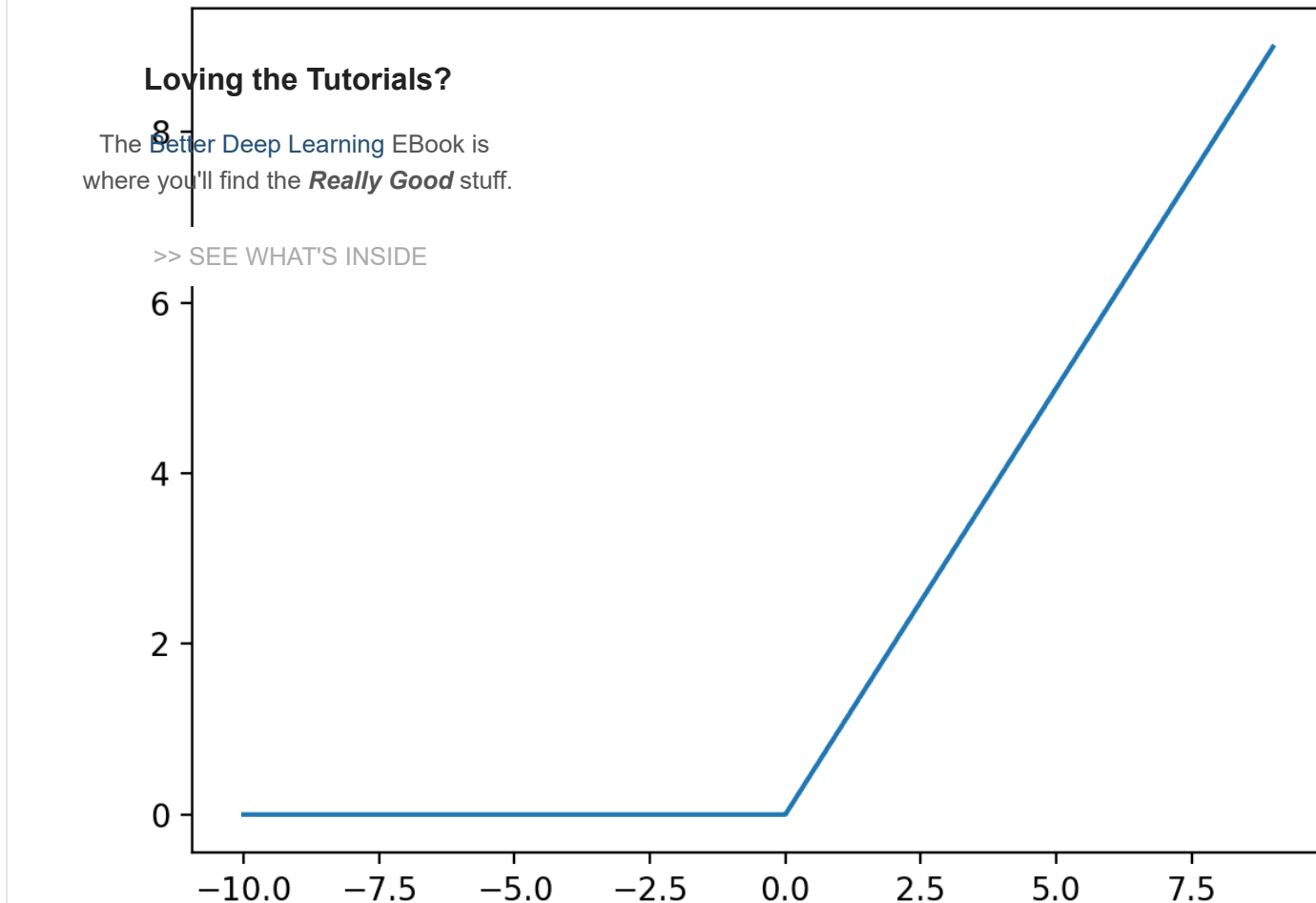
Picked for you:

each input, then plots the result.

```
1 # plot inputs and outputs
2 from matplotlib import pyplot
3
4 # rectified linear function
5 def rectified(x):
6     return max(0.0, x)
7
8 # define a series of inputs
9 series_in = [x for x in range(-10, 11)]
10 # calculate outputs for our inputs
11 series_out = [rectified(x) for x in series_in]
12 # line plot of raw inputs to rectified outputs
13 plt.plot(series_in, series_out)
14 pyplot.show()
```

Running the example creates a line plot showing that all negative inputs are returned as 0, and the positive outputs are returned as-is, resulting in a line plot that is zero for negative inputs and increases linearly for positive inputs (e.g. 1 to 10).

Gentle Introduction to the Adam Optimization Algorithm for Deep Learning



Loving the Tutorials?

The Better Deep Learning EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE

Line Plot of Rectified Linear Activation for Negative and Positive Inputs

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.**

Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Start Machine Learning

The derivative of the rectified linear function is also easy to calculate. Recall that the derivative of the activation function is required when updating the weights of a node as part of the backpropagation of error.



The derivative of the function is the slope. The slope for negative values is 0.0 and the slope for positive values is 1.0.

Picked for you:

Traditionally, the field of neural networks has avoided any activation function that was not completely differentiable, delaying the adoption of the Rectified Linear function and other piecewise-linear functions. Technically, we calculate the derivative when the input is 0.0, therefore, we can assume it is zero. This is not a problem in practice.

[Stacking Ensemble for Deep Learning](#)

[For example, the rectified linear function \$g\(z\) = \max\{0, z\}\$ is not differentiable at \$z = 0\$. This may seem like it invalidates \$g\$ for use with a gradient-based learning algorithm, but in practice, it performs well enough for these models to be used for...](#)

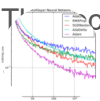
[How To Improve Deep Learning Performance](#)

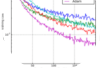
[Page 192, Deep Learning, 2016.](#)

[Using the rectified linear activation function offers many advantages](#)

[How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)

Advantages of the Rectified Linear A

[The rectified linear activation function has rapidly become the de facto standard of neural networks](#)

[Gentle introduction to the Adam Optimization Algorithm for Deep Learning](#)

As such, it is important to take a moment to review some of the benefits of the approach, first highlighted by Xavier Glorot, et al. in their milestone 2012 paper on using ReLU titled “[Deep Sparse Rectifier Neural Networks](#)”.

Loving the Tutorials?

1. Computational Simplicity.

The [Better Deep Learning Book](#) is where you'll find the **Really Good** stuff. The rectifier function is trivial to implement, requiring a *max()* function.

>> SEE WHAT'S INSIDE

This is unlike the tanh and sigmoid activation function that require the use of an exponential calculation.

“Computations are also cheaper: there is no need for computing the exponential function in activations

— [Deep Sparse Rectifier Neural Networks](#), 2011.

2. Representational Sparsity

An important benefit of the rectifier function is that it is capable of outputting a true zero value.

This is unlike the tanh and sigmoid activation functions that learn to approximate a zero output, e.g. a value very close to zero, but not a true zero value.

This means that negative inputs can output true zero values allowing the activation of hidden layers in neural networks to contain one or more true zero values. This is called a sparse representation and is a desirable property in representational learning as it can accelerate learning and simplify the model.

An area where efficient representations such as sparsity are studied and sought is in autoencoders, where a network learns a compact representation of an input (called the code layer), such as an image or series, before it is reconstructed from the compact representation.

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

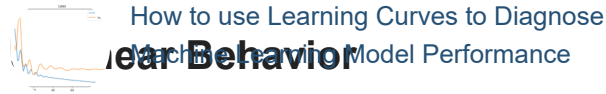
Start Machine Learning

Never miss a tutorial!

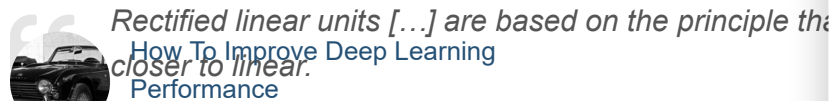
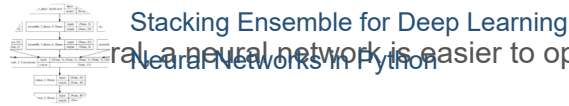
One way to achieve actual zeros in h for sparse (and denoising) autoencoders [...] The idea is to use rectified linear units to produce the code layer. With a prior that actually pushes the representations to zero (like a L1 or a L2 penalty), one can thus indirectly control the average number of zeros in the representation.

Picked for you:

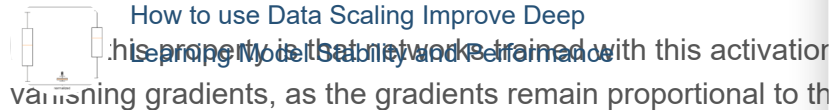
— Page 507, [Deep Learning](#), 2016.



The rectifier function mostly looks and acts like a linear activation function.



— Page 194, [Deep Learning](#), 2016.



— [Deep Sparse Rectifier Neural Networks](#), 2011.

Loving the Tutorials?

4. Train Deep Networks

The [Better Deep Learning](#) Ebook is where you'll find the **Really Good** stuff. Importantly, the (re-)discovery and adoption of the rectified linear activation function meant that it became possible to exploit [>> SEE WHAT'S INSIDE](#) and successfully train deep multi-layered networks with a nonlinear activation function using backpropagation.

In turn, cumbersome networks such as Boltzmann machines could be left behind as well as cumbersome training schemes such as layer-wise training and unlabeled pre-training.

“... deep rectifier networks can reach their best performance without requiring any unsupervised pre-training on purely supervised tasks with large labeled datasets. Hence, these results can be seen as a new milestone in the attempts at understanding the difficulty in training deep but purely supervised neural networks, and closing the performance gap between neural networks learnt with and without unsupervised pre-training.

— [Deep Sparse Rectifier Neural Networks](#), 2011.

Tips for Using the Rectified Linear Activation

In this section, we'll take a look at some tips when using the rectified linear activation function in your own deep learning neural networks.

Use ReLU as the Default Activation Function

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Start Machine Learning

For a long time, the default activation to use was the sigmoid activation function. Later, it was the tanh activation function.



In modern deep learning neural networks, the default activation function is the rectified linear activation function.

Picked for you:

- How to use the introduction of rectified linear units, most neural networks used the logistic sigmoid activation function or the hyperbolic tangent activation function.
- How to use Learning Curves to Diagnose Machine Learning Model Performance
- Page 195, Deep Learning, 2016.

Mapping the Evolution of Deep Learning results will describe a network using ReLU. For example, in the milestone paper by Alex Krizhevsky, et al. titled “ImageNet Classification with Deep Convolutional Neural Networks” the authors developed a deep convolutional neural network with ReLU activation function and achieved top results on the ImageNet photo classification dataset.

- How To Improve Deep Learning Performance
- ... we refer to neurons with this nonlinearity as Rectified Linear Units (ReLU). In practice, neural networks with ReLUs train several times faster

How to use Data Scaling Improve Deep Learning Model Stability and Performance

Doubt, start with ReLU in your neural network, then perhaps compare it to other activation functions and see how their performance compares.

- Gentle Introduction to the Adam Optimization Algorithm for Deep Learning
- In Optimization Algorithms for Deep Learning, the default recommendation is to use the rectified linear unit or ReLU

— Page 174, Deep Learning, 2016.

Loving the Tutorials?
Use ReLU with MLPs, CNNs, but Probably Not RNNs

The Better Deep Learning eBook is available for free. The book can be used as a reference for the different types of neural networks.

It is recommended to use ReLU for both Multilayer Perceptron (MLP) and Convolutional Neural Networks (CNNs).

The use of ReLU with CNNs has been investigated thoroughly, and almost universally results in an improvement in results, initially, surprisingly so.

... how do the non-linearities that follow the filter banks influence the recognition accuracy. The surprising answer is that using a rectifying non-linearity is the single most important factor in improving the performance of a recognition system.

— What is the best multi-stage architecture for object recognition?, 2009

Work investigating ReLU with CNNs is what provoked their use with other network types.

[others] have explored various rectified nonlinearities [...] in the context of convolutional networks and have found them to improve discriminative performance.

— Rectified Linear Units Improve Restricted Boltzmann Machines, 2010.

When using ReLU with CNNs, they can be used as the activation function on the filter maps themselves, followed then by a pooling layer.

Start Machine Learning

You can master applied Machine Learning without math or fancy degrees. Find out how in this free and practical course.

Email Address

START MY EMAIL COURSE

Start Machine Learning

Never miss a tutorial

Typical a convolutional network consists of three stages [...] In the second stage, each linear activation is run through a nonlinear activation function, such as the rectified linear activation function. This is sometimes called the detector stage.

Page 239, [Deep Learning](#), 2016.

Typically, LSTMs use the $\tanh$ activation function for the activation of the cell state and the sigmoid activation function for the node output. Given their careful design, ReLU were thought to not be appropriate for Recurrent Neural Networks (RNNs) such as the Long Short-Term Memory Network (LSTM) by default.

Stacking Ensemble for Deep Learning

At first sight, ReLUs seem inappropriate for RNNs because they can have very large outputs so they might be expected to be far more likely to explode than

How To Improve Deep Learning

A Simple Way to Initialize Recurrent Networks of Rectified

Nevertheless, there has been some work on investigating the result of which is a careful initialization of network weights to

How to use Data Scaling Improve Deep Learning Model Stability and Performance

Try a Smaller Bias Input Value

Gentle Introduction to the Adam Optimization Algorithm for Deep Learning

The bias has the effect of shifting the activation function and it is traditional to set the bias input value to 1.0.

When using ReLU in your network, consider setting the bias to a small value, such as 0.1.

Loving the Tutorials?

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff. *... it can be a good practice to set all elements of [the bias] to a small, positive value, such as 0.1. This makes it very likely that the rectified linear units will be initially active for most inputs in the training set*

— Page 193, [Deep Learning](#), 2016.

There are some conflicting reports as to whether this is required, so compare performance to a model with a 1.0 bias input.

Use “He Weight Initialization”

Before training a neural network, the weights of the network must be initialized to small random values.

When using ReLU in your network and initializing weights to small random values centered on zero, then by default half of the units in the network will output a zero value.

For example, after uniform initialization of the weights, around 50% of hidden units continuous output values are real zeros

— [Deep Sparse Rectifier Neural Networks](#), 2011.

There are many heuristic methods to initialize the weights for a neural network, yet there is no best weight initialization scheme and little relationship beyond general guidelines

Start Machine Learning

the choice of activation function.

Never miss a tutorial:

Prior to the wide adoption of ReLU, Xavier Glorot and Yoshua Bengio proposed an initialization scheme in their 2010 paper titled “Understanding the difficulty of training deep feedforward neural networks” that quickly became the default when using sigmoid and tanh activation functions, generally referred to as “*Xavier initialization*”. Weights and bias values sampled uniformly from a range proportional to the size of the number of nodes in the previous layer (specifically $\pm 1/\sqrt{n}$ where n is the number of nodes in the prior layer).

He, et al. in their 2015 paper titled “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification” suggested that Xavier initialization and other schemes were not appropriate for ReLU and extensions.

Stacking Ensemble for Deep Learning
Neural Networks in Python

Glorot and Bengio proposed to adopt a properly scaled “Xavier” initialization [...]. Its derivation is based on the

assumption is valid for ReLU
Performance

— Delving Deep into Rectifiers: Surpassing Human-Level Per

proposed a small modification of Xavier initialization to n
referred to as “*He initialization*” (specifically $\pm \sqrt{2/n}$ where
the fan-in). In practice, both Gaussian and uniform versions of

Gentle Introduction to the Adam
Initialization Algorithm for Deep Learning

Input Data

It is good practice to scale input data prior to using a neural network.

This may involve standardizing variables to have a zero mean and unit variance or normalizing each value to the scale 0-to-1.

The Better Deep Learning EBook is

where you'll find the **Really Good** stuff.

Without data scaling on many problems, the weights of the neural network can grow large, making the network unstable ; initialization error.

>> SEE WHAT'S INSIDE

This good practice of scaling inputs applies whether using ReLU for your network or not.

Use Weight Penalty

By design, the output from ReLU is unbounded in the positive domain.

This means that in some cases, the output can continue to grow in size. As such, it may be a good idea to use a form of weight regularization, such as an **L1 or L2 vector norm**.

Another problem could arise due to the unbounded behavior of the activations; one may thus want to use a regularizer to prevent potential numerical problems. Therefore, we use the L1 penalty on the activation values, which also promotes additional sparsity

— Deep Sparse Rectifier Neural Networks, 2011.

This can be a good practice to both promote sparse representations (e.g. with L1 regularization) and reduced generalization error of the model.

Extensions and Alternatives to ReLU

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Start Machine Learning

The ReLU does have some limitations.

Key among the limitations of ReLU is the case where large weight updates can mean that the summed input to the activation function is always negative, regardless of the input to the network.

This means that a node with this problem will forever output an activation value of 0.0. This is referred to as a “dying ReLU”.

How to use Learning Curves to Diagnose Machine Learning Model Performance
the gradient is 0 whenever the unit is not active. This could lead to cases where a unit never activates as a gradient-based optimization algorithm will not adjust the weights of a unit that never activates initially. Further, like the vanishing gradients problem, we might expect learning to be slow when training ReL networks with constant 0 gradients.

Rectifier Nonlinearities Improve Neural Network Acoustic Models, 2013.
How To Improve Deep Learning Performance
Popular extensions to the ReLU relax the non-linear output in some way.

How to use Data Scaling Improve Deep Learning Model Stability and Performance
Leaky ReLU (LReLU or LREL) modifies the function to allow a small, non-zero gradient when the unit is not active.
The leaky rectifier allows for a small, non-zero gradient when the unit is not active.

Rectifier Nonlinearities Improve Neural Network Acoustic Models, 2013.

The Exponential Linear Unit or ELU, is a generalization of the ReLU that uses a parameterized exponential function to transition from the positive to small negative values.

The Better Deep Learning EBook is where you'll find the Really Good stuff.
ELUs have negative values which pushes the mean of the activations closer to zero. Mean activations are closer to zero, which leads to faster learning as they bring the gradient closer to the natural gradient.

Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs), 2016.

The Parametric ReLU, or PReLU, learns parameters that control the shape and leaky-ness of the function.

... we propose a new generalization of ReLU, which we call Parametric Rectified Linear Unit (PReLU). This activation function adaptively learns the parameters of the rectifiers

Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, 2015.

Maxout is an alternative piecewise linear function that returns the maximum of the inputs, designed to be used in conjunction with the dropout regularization technique.

We define a simple new model called maxout (so named because its output is the max of a set of inputs, and because it is a natural companion to dropout) designed to both facilitate optimization by dropout and improve the accuracy of dropout's fast approximate model averaging technique.

Maxout Networks, 2013.

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

Email Address

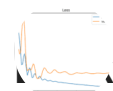
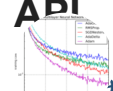
START MY EMAIL COURSE

Start Machine Learning

Further Reading

Use the social media links below to find more resources on the topic if you are looking to go deeper.

Posts Picked for you:

- How to Fix Vanishing Gradients Using the Rectified Linear Activation Function
 How to use Learning Curves to Diagnose Machine Learning Model Performance
- Section 6.3.1 Rectified Linear Units and Their Generalizations, [Deep Learning](#), 2016.
 Stacking Ensemble for Deep Learning
 Neural Networks in Python
- What is the best multi-stage architecture for object recognition?
 Rectified Linear Units Improve Restricted Boltzmann Machines
 Deep Sparse Rectifier Neural Networks, 2011.
- Rectifier Nonlinearities Improve Neural Network Acoustic Performance
- Understanding the difficulty of training deep feedforward networks
 How to use Data Scaling to Improve Deep Learning Model Stability and Performance
 Living Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification with AlexNet
 About Networks, 2013.
- Gentle Introduction to the Adam Optimization Algorithm for Deep Learning
 Max API

Articles

- [Loving the Tutorials?](#)
- [Neural Network FAQ](#)
- [Activation function](#). Wikipedia
- [The Better Deep Learning EBook is](#)
- [Vanishing gradient problem](#). Wikipedia.
- [Why is my gradient really 0?](#)
- [Rectifier \(neural networks\)](#). Wikipedia.
- [Piecewise linear](#). Wikipedia.

Summary

In this tutorial, you discovered the rectified linear activation function for deep learning neural networks.

Specifically, you learned:

- The sigmoid and hyperbolic tangent activation functions cannot be used in networks with many layers due to the vanishing gradient problem.
- The rectified linear activation function overcomes the vanishing gradient problem, allowing models to learn faster and perform better.
- The rectified linear activation is the default activation when developing multilayer Perceptron and convolutional neural networks.

Do you have any questions?

Ask your questions in the comments below and I will do my best to answer.

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

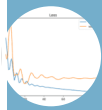
Never miss a tutorial:



Better Deep Learning

Train Faster, Reduce Overfitting,
and Make Better Predictions

Picked for you:

- 

Jason Brownlee
How to use Learning Curves to Diagnose Machine Learning Model Performance
- 

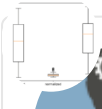
MACHINE LEARNING MASTERY
Stacking Ensemble for Deep Learning Neural Networks in Python
- 

How to Improve Deep Learning Performance

Tweet

Share

Share



How to use Data Scaling Improve Deep Learning Model Stability and Performance



About Jason Brownlee
Jason Brownlee, PhD is a machine learning specialist in machine learning methods via hands-on tutorials.
View all posts by Jason Brownlee →

Train Faster, Reduce Overfitting, and Ensembles

...with just a few lines of python code

Discover how in my new Ebook:
[Better Deep Learning](#)

It provides **self-study tutorials** on topics like:
weight decay, batch normalization, dropout, model stacking and much more...

Bring better deep learning to your projects!

Skip the Academics. Just Results.

SEE WHAT'S INSIDE

Start Machine Learning

✕

You can master applied Machine Learning **without math or fancy degrees.**
Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

< Ensemble Neural Network Model Weights in Keras (Polyak Averaging) How to Fix the Vanishing Gradients Problem Using the ReLU >

Loving the Tutorials?

The [Better Deep Learning](#) Ebook is
where you'll find the **Really Good** stuff.

65 Res, >> SEE WHAT'S INSIDE *Introduction to the Rectified Linear Unit (ReLU)*



Vaibhav Singh January 10, 2019 at 5:07 pm #

REPLY ↩

This was a great explanation. Thank you.



Jason Brownlee January 11, 2019 at 7:41 am #

REPLY ↩

Thanks, I'm glad it helped.



Vikram January 11, 2019 at 1:34 pm #

REPLY ↩

Excellent explanation and supplemented well with additional resources. Thank you, Jason.

Jason Brownlee January 12, 2019 at 5:34 am #

Start Machine Learning

✕

You can master applied Machine Learning **without math or fancy degrees.**
Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Never miss a tutorial: I'm glad it helped.

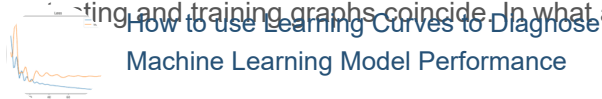


 **Linu Pinto** January 11, 2019 at 2:45 pm #

REPLY ↩

Picked for you:

How can we analyse the performance of nn. Is it when mean squared error is minimum and validation training and training graphs coincide. In what all ways we can analyse it's performance.



 **Jason Brownlee** January 12, 2019 at 5:36 am #

REPLY ↩

[Neural Networks in Python](#)

It really depends on the specifics of the problem.



[How To Improve Deep Learning Performance](#)

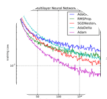


Smit Shah January 14, 2019 at 9:01 pm #

[How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)

What will happen if we do the other way round? I mean

output 0 for positive values.



[Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

 **Jason Brownlee** January 15, 2019 at 5:51 am #

REPLY ↩

Loving the Tutorials?

Probably poor results, e.g. we are summing weighted inputs before passing through the activation. It would encourage negative weighted sums I guess. Nevertheless, try it and see what happens.

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff.



>> SEE WHAT'S INSIDE [in](#) April 19, 2019 at 9:47 am #

REPLY ↩

Please tell me whether relu will help in the problem of detecting an audio signal in a noisy environment. I have already purchased all your books



Jason Brownlee April 19, 2019 at 3:03 pm #

REPLY ↩

It might. Try it and see.



Maltev June 19, 2019 at 12:02 am #

REPLY ↩

Nice introduction, thanks!

I'm taking an online class on deep learning. I read your post and implemented He initialization, before I got to the course material covering it.

Now, I think you have a typo in that equation, you write " $2/\sqrt{n}$ " and it should be $\sqrt{2/n}$, correct?

Start Machine Learning

Never miss a tutorial:

Jason Brownlee

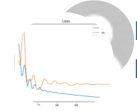
June 19, 2019 at 8:11 am #

REPLY ↩



are correctly. Thanks!

Picked for you:



Jiang

August 8, 2019 at 5:32 pm #

REPLY ↩

How to use Learning Curves to Diagnose
Machine Learning Model Performance
Great post.



Stacking Ensemble for Deep Learning

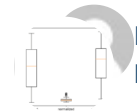
Neural Networks in Python

Jason Brownlee

August 9, 2019 at 8:07 am #



Thanks, I'm glad it helped.
How To Improve Deep Learning
Performance



Felipe Melo

August 29, 2019 at 1:32 am #

How to use Data Scaling Improve Deep
Learning Model Stability and Performance
The use of smooth functions like sigmoid and tanh is for
theory, learn any pattern. By passing the same value for the po
“guaranty”?
Gentle Introduction to the Adam
Optimization Algorithm for Deep Learning

Start Machine Learning



You can master applied Machine Learning
without math or fancy degrees.
Find out how in this *free* and *practical* course.

START MY EMAIL COURSE



Jason Brownlee

August 29, 2019 at 6:14 am #

REPLY ↩

Loving the Tutorials?

No. We gain flexibility.
The [Better Deep Learning](#) EBook is
where you'll find the **Really Good** stuff.



>> SEE WHAT'S INSIDE

September 19, 2019 at 1:44 am #

REPLY ↩

If you think about it you end up with a switched system of linear projections.

For a particular input and a particular neighborhood around that input a particular linear projection from the input to the output is in effect. Until the change in the input is large enough for some switch (ReLU) to flip state. Since the switching happens at zero no sudden discontinuities in the output occur as the system changes from one linear projection to the other.



Jason Brownlee

September 19, 2019 at 6:03 am #

REPLY ↩

That's a nice way to think about it Sean, thanks.



Sean O'Connor

September 26, 2019 at 8:09 am #

REPLY ↩

No problem. Just for some people who don't get it – when a switch is on 1 volt in will give 1 volt out, n volts in will give n volts out. Which gives you a 45 degree line when you graph it out. When it is off you get zero volts out, a flat line.

ReLU is then a switch with its own decision making pol

Start Machine Learning

Never miss a tutorial: The weighted sum of a number of weighted sums is still a linear system.

A ReLU neural network is then a switched system of weighted sums of weighted sums of....

There are no "continuities" during switching for gradual changes of the input because switching happens at zero.

For a particular input and a particular output neuron the output is a linear composition of weighted sums that can be converted to a single weighted sum of the input. Maybe you can look at that weighed sum to see

what the neural network is looking at in the input. Or there are metrics you can calculate like the angle between the input vector and the weight vector of the final weighed sum.

Machine Learning Model Performance

Stacking Ensemble for Deep Learning

René Steiner September 25, 2019 at 5:14 pm #

Great tutorial, thanks a lot.

How To Improve Deep Learning

Performance

minor point: I think the explanation you give here is rather because the rectified function is linear for half of the input domain a piecewise linear function or a hinge function."

How to use Data Scaling Improve Deep

Learning Model Stability and Performance

properties of linearity (see e.g. <https://en.wikipedia.org/wiki/>

Gentle Introduction to the Adam

Optimization Algorithm for Deep Learning

Obviously, the rectified function is linear in both halves of the input and homogeneity of degree 1:

$f(\alpha x) = \alpha f(x)$ for all α .

Loving the Tutorials?

(This doesn't hold for $\alpha < 0$.)

The Better Deep Learning EBook is

where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE September 26, 2019 at 6:31 am #

Excellent, thanks for pointing this out!

Sylvia October 13, 2019 at 10:56 am #

how to predict the output Y with the input X using SPSS neural network> I do not understand how to use the model to predict the output. How to calculate the value of Y with the certain value of X.

thank you.

Jason Brownlee October 14, 2019 at 8:02 am #

Sorry, I don't know about SPSS.

Truong Phuoc Hieu December 13, 2019 at 8:41 pm #

Thanks for this concise explanation. 😊 Bookmarked

Start Machine Learning

You can master applied Machine Learning

without math or fancy degrees.

Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Start Machine Learning

Never miss a tutorial:

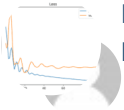


December 14, 2019 at 6:16 am #

REPLY ↩

You're welcome, I'm happy it helped!

Picked for you:



How to use Learning Curves to Diagnose Machine Learning Model Performance

Sarfraz Equbal December 27, 2019 at 3:36 am #

REPLY ↩



hello Jason,

Stacking Ensemble for Deep Learning

you help me with the name of original research paper of ReLU. thanks in advance.

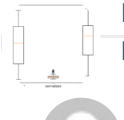
Neural Networks in Python



How To Improve Deep Learning performance

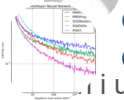
Jason Brownlee December 27, 2019 at 6:37 am #

See the “further reading” section of the tutorial.



How to use Data Scaling Improve Deep Learning Model Stability and Performance

Jesse January 30, 2020 at 11:22 pm #



Gentle Introduction to the Adam Optimization Algorithm for Deep Learning

Hello Jason,

i use ReLU on autoencoders?

Start Machine Learning ✕

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

START MY EMAIL COURSE



Loving the Tutorials?

Jason Brownlee January 31, 2020 at 7:51 am #

Better Deep Learning Ebook is where you'll find the **Really Good** stuff.

In the hidden layer, you can.

REPLY ↩

>> SEE WHAT'S INSIDE



Devc February 2, 2020 at 10:20 pm #

Nicely narrated.

REPLY ↩



Jason Brownlee February 3, 2020 at 5:45 am #

Thanks!

REPLY ↩



Yogeeshwari February 20, 2020 at 10:14 pm #

A very crisp and clear explanation!

REPLY ↩



Jason Brownlee February 21, 2020 at 8:21 am #

Thanks!

REPLY ↩

Start Machine Learning

Never miss a tutorial:



REPLY ↩

As a person who was heavily involved in the early days of backprop but away from the field for many years, I have several problems with the ReLu method. Perhaps you could explain them away.

1 The ReLu method makes the vanishing gradient problem MUCH WORSE, since for all negative values the derivative is precisely zero. Yes, you mention ways to remedy this, but they aren't ReLu.
MLPs can approximate arbitrary functions only because of the nonlinearity of the activation function (otherwise they degenerate into single layer perceptrons). Here the nonlinearity is concentrated at a single point – every else activation is linear. How much expressivity is sacrificed?
backprop relies on derivatives being defined – ReLu's derivative at zero is undefined (I see people use zero there, which is the derivative from the left and thus a valid subderivative for backproping)

As you mentioned (but this is not "magic", it can be shown to converge to the highest eigenvalue of the Hessian), converge with zero average and tanh activation, rather than [0,1] and log
Thanks

How to use Data Scaling Improve Deep Learning Model Stability and Performance

 **Jason Brownlee** February 27, 2020 at 6:00 am #
A little Introduction to the Adam Optimization Algorithm for Deep Learning
Thanks for sharing your concerns with ReLU. Perhaps the results achieved with relu instead of sigmoid/tanh speak for themselves?

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.**
Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Loving the Tutorials?

 **Robin** March 26, 2020 at 4:05 am #
The Better Deep Learning eBook is where you'll find the **Really Good** stuff.
Hi jason,

REPLY ↩

perhap >> SEE WHAT'S INSIDE s post but could you please tell us how the network actually learns? i mean lets say we have a image of a circle so how is the image recognized? i mean, what would be the desired output that we are trying to match here and how does the entire process work out? many thanks for a nice post on activation functions.

 **Jason Brownlee** March 26, 2020 at 8:01 am #
Good question, perhaps start with this tutorial:
<https://machinelearningmastery.com/tutorial-first-neural-network-python-keras/>

REPLY ↩

 **Radhika Gopinath** May 15, 2020 at 1:09 pm #
Very good progressive explanation!

REPLY ↩

 **Jason Brownlee** May 15, 2020 at 1:30 pm #
Thanks!

REPLY ↩

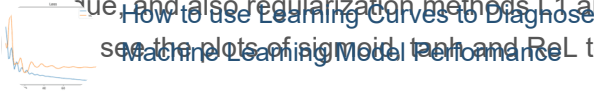
Start Machine Learning

Never miss a tutorial:



REPLY ↩

Picked for you: Wow, very nice tutorial, thank you very much. This really helps people who have begun learning about ANNs, etc. like me. My only complaint is that explanations of the disadvantages of the sigmoid and tanh were a little brief, and also regularization methods L1 and L2 were not described, at least briefly. Also, it would be really nice to see the plots of sigmoid, tanh and ReL together to compare and contrast them.



Stacking Ensemble for Deep Learning

Jason Brownlee June 9, 2020 at 6:07 am #

REPLY ↩

Thanks.

How To Improve Deep Learning Performance

How to use Data Scaling Improve Deep Learning Model Stability and Performance

Abhishek June 30, 2020 at 4:54 am #

Hi Jason,

Thanks for this explanation. Awesome article. I came across on "Optimization Algorithms for Deep Learning" and the "Deactivation state" and "Noise robustness" like LReLU or L2L2, I've read that "Deactivation state is not noise robust".

Can you please explain this concept?Also, what is meant by "Deactivation state" and "Noise robustness"?

Loving the Tutorials?

The Better Deep Learning EBook is where you'll find the Really Good stuff

Jason Brownlee June 30, 2020 at 6:30 am #

REPLY ↩

>> SEE WHAT'S INSIDE

Perhaps ask the authors of the explanation you "came across"?

Gulun July 5, 2020 at 5:41 am #

REPLY ↩

Hi Jason,

Good Morning

Could you please let me know your expert advice on below question.

In SIGMOID activation function, It's output is either 0 or 1 and when it is zero then respective neuron will not be activated. Then why SIGMOID is not sparse activation function.

Thanks

Jason Brownlee July 5, 2020 at 7:07 am #

REPLY ↩

No, sigmoid output is a value between 0 and 1, not 0 or 1

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Never miss a tutorial:

Gulun

July 5, 2020 at 12:29 pm #

REPLY ↩

Hi Jason,

Thanks for your reply.

Picked for you SIGMOID range is between 0 and 1. But it's out put can be 0 . In that case it will be sparse.

Please let me know your advice

How to use Learning Curves to Diagnose Machine Learning Model Performance

Stacking Ensemble

Jason Brownlee

July 6, 2020 at 6:25 am #

Neural Networks in Python

Yes, it can be zero and if you have many z

REPLY ↩

How To Improve Deep Learning Performance

LearnD

July 5, 2020 at 10:09 pm #

In SIGMOID Activation Function, if the output is less t

How to use Data Scaling Improve Deep Learning Model Stability and Performance

the respective Stability and Performance. Then I think Ne

Explain.

How approximate to "0" is not Sparse?

Gentle Introduction to the Adam

Optimization Algorithm for Deep Learning

Start Machine Learning

✕

You can master applied Machine Learning **without math or fancy degrees.**

Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Loving the Tutorials?

Jason Brownlee

July 6, 2020 at 6:34 am #

The Better Deep Learning EBook is

It depends on the nature of the input.

where you'll find the **Really Good** stuff.

REPLY ↩

>> SEE WHAT'S INSIDE

Katta Venkata RAMANA

August 16, 2020 at 12:43 am #

REPLY ↩

AttributeError: 'Activation' object has no attribute 'get_output'. This is in case of the following:

```
output_layer = model.layers[1].get_output()
output_fn = theano.function([model.layers[0].get_input()], output_layer)
```

Can you tell what is substitue for attribute model.layers.get_output()

Jason Brownlee

August 16, 2020 at 5:54 am #

Sorry, I have not seen this issue before. Perhaps try posting to stackoverflow?

REPLY ↩

Adem Aldemir

September 22, 2020 at 1:50 am #

Awesome explanation! 🍌

REPLY ↩

Start Machine Learning

Never miss a tutorial:

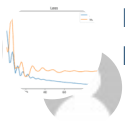


September 22, 2020 at 6:52 am #

REPLY ↩

Well done!

Picked for you:



[How to use Learning Curves to Diagnose Machine Learning Model Performance](#)

Tanuja Shrestha October 24, 2020 at 7:17 am #

REPLY ↩

Hi Jason,

[Stacking Ensemble for Deep Learning](#)

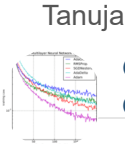
[Neural Networks in Python](#)

Recently, I learned a regression problem that used sigmoid function. The solution was fundamentally expressed in a fraction from 0 to 1. The sigmoid function was used.

[How To Improve Deep Learning Performance](#)

The solution did not use that 0.5 threshold to get the binary probabilities value. And, I understood this part well. Also, the regression function was used.

Now, the confusing part is, yet, it used mean squared error as a cost function. I read that log loss is the cost function for the sigmoid activation function. The question is: what could have done things right in the case of a regression problem.



Tanuja

[Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

REPLY ↩

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE



Jason Brownlee October 24, 2020 at 7:54 am #

Loving the Tutorials?

REPLY ↩

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff.



Tanuja Shrestha October 24, 2020 at 8:29 am #

>> SEE WHAT'S INSIDE

Hi Jason, I did not get the meaning. Do you mean that the model is not rational? Thanks!

REPLY ↩



Jason Brownlee October 24, 2020 at 12:41 pm #

Sorry, I meant that the model sounds fine to me.

REPLY ↩



Tanuja Shrestha November 10, 2020 at 10:21 pm #

Thanks Jason.

REPLY ↩

Can you give more explanation on why using mse instead of the log loss metric is still okay in the above-described case?

On my search on the Internet, I found that sigmoid with log loss metric penalizes the wrong predicted classes more than the mse metric.

So, can I understand that the very fact that we are interested in knowing the values between 0 to 1 only, not the two classes, justifies the use of mse metric?

Start Machine Learning

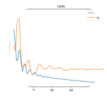
Any explanation is appreciated.
Never miss a tutorial:



Jason Brownlee November 11, 2020 at 6:47 am #

REPLY ↩

Picked for you



[How to use Learning Curves to Diagnose](#)

[Machine Learning Model Performance](#)

Perhaps try it and compare result to a linear activation.

Learn more about loss functions here:



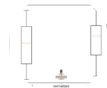
<https://machinelearningmastery.com/how-to-choose-loss-functions-when-training-deep-learning-neural-networks/> in Python



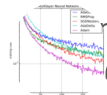
How To Improve Deep Learning

Performance November 16, 2020 at 7:34 am #

Hi Jason,



[How to use Data Scaling Improve Deep Learning Model Stability and Performance](#) ways, excellent post! I learned so much from you 😊



I have a question about activation functions and LSTMs (I'm trying to use them for classification). LSTM units internally already have 4 neural layers, so I don't think it's necessary to have activation functions (tanh) are present within the units? On a lot of blogs/tutorials I find people using ReLu for LSTMs but I was wondering why? As the units outputs a multiplication between sigmoid and tanh, is it not weird to use a ReLu after that? Also, LSTM do not struggle with vanishing gradient so I do not understand the advantage of using it. The references you mention use RNN with ReLu and not LSTM so I did not find my answer there.

[Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#) necessary for LSTM networks to have activation functions (tanh) are present within the units? On a lot of blogs/tutorials I find people using ReLu for LSTMs but I was wondering why? As the units outputs a multiplication between sigmoid and tanh, is it not weird to use a ReLu after that? Also, LSTM do not struggle with vanishing gradient so I do not understand the advantage of using it. The references you mention use RNN with ReLu and not LSTM so I did not find my answer there.

Loving the Tutorials?

In another blog (<https://machinelearningmastery.com/how-to-develop-rnn-models-for-human-activity-recognition-time-series-the-basics/>) you did not specify any activation for the LSTM, meaning you went with Keras' default. Any reason you used this default (activation=tanh, recurrent activation=sigmoid) or do you know the reason why this is the default in the first place instead of no (recurrent) activation function?

>> SEE WHAT'S INSIDE

Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees.**

Find out how in this *free* and *practical* course.

START MY EMAIL COURSE



Jason Brownlee November 16, 2020 at 7:37 am #

REPLY ↩

Thanks!

The defaults are based on the original LSTM paper and work well in many cases.

I find using relu activations in LSTMs useful for some times series problems.

Perhaps try each and compare for your specific dataset and use what works best / lowest error.



Laura November 21, 2020 at 2:10 am #

REPLY ↩

Thank you so much! I'll try ReLu and the defaults. Just one more thing, to be sure that I understand what you're saying. I've been searching all over the internet so it would be very helpful if you know the following 😊

You say that the defaults are based on the original paper. Does the recurrent_activation in Keras (sigmoid) then denote the sigmoids that are used by the gates in the LSTM cell. And does the activation in Keras (tanh) denote the tanh through which the cell state goes?

Start Machine Learning

outputted? If this is true, then changing the default Keras activation thus changes the original architecture of the LSTM cell itself as designed by Hochreiter? So if I want to use the LSTM as designed originally I should

Never miss a tutorial: NOT change the functions in Keras (and thus use Keras' defaults)? Or is this incorrect and do the activation functions in Keras add an ADDITIONAL activation on top of the originally designed LSTM cell (with its sigmoids and tanhs included)?

Picked for you: Hope you can help me!!!

Best How to use Learning Curves to Diagnose Machine Learning Model Performance

Stacking Ensemble for Deep Learning Neural Networks in Python Correct. By default it is the standard LSTM the standard LSTM.

How To Improve Deep Learning Performance

How to use Data Scaling Improve Deep Learning Model Stability and Performance Great! Thanks 😊

Gentle Introduction to the Adam Optimization Algorithm for Deep Learning You're welcome.

Loving the Tutorials?

The Better Deep Learning EBook is where you'll find the Really Good stuff.

Leave a Reply

>> SEE WHAT'S INSIDE

Name (required)

Email (will not be published) (required)

Website

SUBMIT COMMENT

REPLY ↩

Start Machine Learning

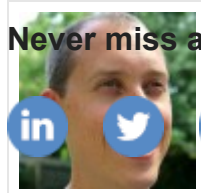
You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Start Machine Learning

Never miss a tutorial:



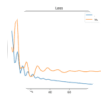
Welcome!

I'm Jason Brownlee PhD

and I help developers get results with machine learning.



Picked for you:



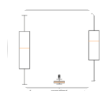
How to use Learning Curves to Diagnose Machine Learning Model Performance



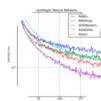
Stacking Ensemble for Deep Learning Neural Networks in Python



How To Improve Deep Learning Performance



How to use Data Scaling Improve Deep Learning Model Stability and Performance



Gentle Introduction to the Adam Optimization Algorithm for Deep Learning

Loving the Tutorials?

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE

Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees.**

Find out how in this *free* and *practical* course.

START MY EMAIL COURSE